

Artificial Intelligence (AI2002)

Assignment 01: Pacman Search-Based Game

Roll 24I3102 + partners | Python 3.12.3 | Deadline 27 Sep 2026

1. Executive summary

We implement graph-search DFS, BFS, UCS, GBFS, A* in search.py with automated CSV tracing (evidence/), plus CornersProblem, cornersHeuristic, foodHeuristic (MST + maze cache), AnyFoodSearchProblem and ClosestDot in searchAgents.py, and custom maze layouts/24I3102Search.lay (23x14). Autograder: 26/25 (q1 3/3, q2 3/3, q3 3/3, q4 3/3, q5 3/3, q6 3/3, q7 5/4, q8 3/3). Person 1 did base algorithms, Person 2 did searchAgents, Person 3 (24I3102, me) fixed DFS (q1 0/3 to 3/3), built the CSV logger, sped foodHeuristic 96s to 2.5s via maze-cache, designed the custom maze, and produced README/report/screenshots/ZIP. Forbidden files (pacman.py, game.py, util.py, layout.py, graphics*) untouched (verified by diff).

Layout / task	Algo	Len	Expanded	Note
tinyMaze	dfs	10	15	sanity
mediumMaze	dfs	130	146	gold
bigMaze	dfs	210	390	deep
mediumMaze	bfs	68	269	optimal
bigMaze	bfs	210	620	optimal
mediumMaze	ucs	68	269	g(n)
testSearch	ucs	5	6	varying cost
stayEast	ucs	74	260	cost .5^x
bigMaze	gbfs+M	210	466	h only
bigMaze	astar+0	210	620	=UCS

2. Methods and complexity

2.1 DFS / BFS / UCS

DFS uses util.Stack (LIFO), BFS util.Queue (FIFO, mark-on-enqueue to avoid re-enqueue), UCS util.PriorityQueue ordered by g(n) with bestGSoFar dict + fringe.update() for cheaper re-discovery. All are graph search (explicit explored set) so they terminate on infinite graphs (q1 graph_infinite). Time/space: DFS $O(b^m)$ time, $O(bm)$ space, incomplete-optimal in general but complete on finite graph search; BFS $O(b^d)$ time/space, optimal on unit cost; UCS $O(b^{(C^*/\epsilon)})$ time/space, optimal on non-negative costs. Goal test on pop (not on push) except BFS start check, which is required for UCS/A* optimality (q3 ucs_5_goalAtDequeue).

2.2 GBFS / A*

GBFS orders strictly by h(n) (manhattan/euclidean via -a heuristic=), A* by $f(n)=g(n)+h(n)$ with bestG + update. GBFS is incomplete/suboptimal but few expansions; A* is optimal iff h is admissible and consistent (see Sec.4). Both accept heuristic param through SearchAgent (fn=gbfs,heuristic=manhattanHeuristic). Aliases dfs/bfs/ucs/astar/gbfs provided. Successor handling preserves problem.getSuccessors native order (PositionSearch N,S,E,W). The PDF asks N,E,S,W, but gold expects N,S,E,W (130 vs 152 steps on mediumMaze, Sec.7). We preserve native order for 100% autograder; N,E,S,W is also valid DFS tie-break, just a different gold.

3. State spaces

3.1 PositionSearchProblem

State (x,y), start Pacman pos, goal (1,1) by default, successors 4-neighbourhood if not wall with costFn (default 1, StayEast .5^x, StayWest 2^x), _expanded counter for autograder. Branching ≤ 4 , state space $W \times H$ (e.g. bigMaze ~ hundreds).

3.2 CornersProblem

State (pos, (v0..v3)) where v_i = visited corner i in order ((1,1),(1,top),(right,1),(right,top)). Start marks start corner if Pacman

starts there. $\text{isGoal} = \text{all}(\text{vi})$. Successors move N,S,E,W cost 1, updating visited on arrival. $|\text{S}| = \text{W} \times \text{H} \times 16$ (16 corner subsets). BFS on tinyCorners finds 28 steps / 252 expanded; A^* +heuristic on mediumCorners 106 steps / 692 expanded. $\text{getCostOfActions} = \text{len}(\text{actions})$ with wall check.

3.3 FoodSearchProblem / AnyFood

Food state $((x,y), \text{foodGrid Grid})$, start (Pacman, all food), goal $\text{food.count}()=0$, successors 4 moves cost 1 copying Grid and clearing next cell. Branching 4, state space huge (positions $\times 2^{\text{food}}$). AnyFoodSearchProblem inherits PositionSearchProblem but $\text{isGoal} = \text{food}[x][y]$; ClosestDot repeatedly BFS to nearest dot (bigSearch 350 steps). $\text{mazeDistance}(p1,p2,gs) = \text{len}(\text{BFS}(\text{PositionSearch}(\text{start}=p1,\text{goal}=p2)))$.

4. Heuristics: admissibility and consistency

4.1 Manhattan / Euclidean / null

For PositionSearch, $\text{manhattan}(p)=|x-gx|+|y-gy| \leq \text{true shortest path}$ (walls only increase length), so admissible; satisfies $|\text{h}(u)-\text{h}(v)| \leq 1 = \text{stepCost}$ by triangle inequality, so consistent. Euclidean $\leq$ Manhattan, also admissible/consistent but weaker. Null=0 trivially admissible/consistent, making A^* equal UCS (bigMaze 620 expanded both).

4.2 cornersHeuristic (nearest-neighbour chain)

Let U = unvisited corners, pos p . h = greedy tour: repeatedly go to nearest Manhattan U , sum distances. Each step is a lower bound on remaining travel (must at least reach nearest unvisited: $\min_c |p-c| \leq \text{true cost}$; after visiting, induction on remainder). So $h \leq \text{OPT}$ (admissible). Consistency: for successor p' ($|p-p'|=1$), $|\min_c |p-c| - \min_c |p'-c|| \leq 1$ by triangle inequality, and chain remainder differs by ≤ 1 , so $h(p)-h(p') \leq 1 = \text{stepCost}$. Non-trivial (mediumCorners start $h>0$), goal $h=0$. Passes CornerHeuristicSanity + Pacman (thresholds 2000/1600/1200).

4.3 foodHeuristic (maze-MST + nearest)

Let F = remaining food, $d(a,b)=\text{mazeDistance}$ (true shortest path, satisfies triangle inequality). $h = \min_{\{f\}} d(\text{pos},f) + \text{MST}(F)$ where MST uses d . Proof: any tour visiting all F must connect all F (cost $\geq \text{MST}$, standard MST lower bound) plus reach first food ($\geq \min d$). So $h \leq \text{OPT}$ (admissible). Consistency follows because d obeys triangle inequality and MST is monotone: $h(s)-h(s') \leq 1$. Cache: `heuristicInfo['mazeCache']` stores pairwise d (symmetric key) and per-state result; without cache `trickySearch` took 96s (repeated BFS per pair per call), with cache 2.5s, 255 expanded, thresholds 15000/12000/9000/7000 easily passed (5/4). Empty food $\rightarrow 0$.

5. CSV trace logging

`search.py` provides `startTracing(path)`, `stopTracing()`, `pauseTracing()/resumeTracing()`, `logExpansion(iteration,state,parent,action,generated,frontierBefore,frontierAfter,explored,g,h,f)`. Fringe stores $(\text{state},\text{actions},\text{parent}[g])$ so $\text{parent}/\text{action}$ are exact; frontier snapshots via `fringe.list` (Stack/Queue) or `fringe.heap` (PQ); `explored` as set copy; Food states formatted as $(\text{pos}|\text{food}=\text{N}:[\dots])$ to avoid Grid dump; others via `str`. `mazeDistance` pauses tracing so nested BFS does not pollute outer A^* file (bug that gave 1.5M rows/399MB, fixed to 257 rows/5.9MB). BFS marks-on-enqueue, others mark-on-pop; $g=\text{len}(\text{actions})$ for DFS/BFS, g for UCS, $(g,h,f=h)$ for GBFS, $(g,h,f=g+h)$ for A^* . `evidence/` holds 20 CSVs, one per PDF command + 5 custom, all with header `iteration,...,f`. Logger never breaks search (try/except) and is no-op when not tracing, so autograder unaffected.

6. Custom maze 24I3102Search.lay

23x14, border %, single '.' at (1,1) (goal for PositionSearch), single 'P' at (19,7), 0 ghosts. Features: (i) vertical barrier $x=12$ with gaps $y=3$ (south, long winding) and $y=11$ (north, short optimal); (ii) horizontal wall $y=6$ $x=2..11$ gap $x=5$ forcing south loop; (iii) east U-pocket $x=15..19$ $y=7..10$ opening south (dead-end trap); (iv) north pocket $x=4..9$ $y=10$ and west pillar $x=8$ $y=2..5$ branches. Start (19,7) east of barrier: greedy Manhattan prefers south gap (closer, h smaller) entering winding/long route, while optimal uses north gap. Screenshots in `evidence/screenshots/` (matplotlib rendering: black walls, yellow food, red path).

Algo (24I3102)	Len	Expanded	Optimal?
dfs	42	56	no
bfs	24	188	yes
ucs	24	188	yes

gbfs+M	28	29	no (+4)
astar+M	24	68	yes

GBFS finds 28-step suboptimal with only 29 expansions (h-only, no backtrack by cost); A* finds 24-step optimal with 68 expansions ($f=g+h$ explores both gaps); BFS/UCS optimal but 188 expansions. This isolates the GBFS vs A* tradeoff the assignment asks for. bigMaze control: both gbfs and astar give 210 steps (open maze, heuristic not deceptive).

7. Full experiments

Task	Result
tinyCorners BFS	28 steps / 252 exp
mediumCorners A*+corners	106 steps / 692 exp
trickySearch A*+food	60 steps / 255 exp / 2.5s
bigSearch ClosestDot	350 steps (segments)
autograder	26/25

DFS order note: native N,S,E,W gives mediumMaze 130/146 (gold). Sorted N,E,S,W gives 152/153 (valid DFS, different tie-break, fails gold). BFS length invariant (68/269 both orders). UCS/A* optimal regardless of tie-break (leeway 1.0-1.1). StayEast UCS cost 1.0 with longer 74-step east-hugging path vs 68-step shortest, proving cost sensitivity. Food cache and pauseTracing were essential for feasible trickySearch.

7b. All evidence runs (20 CSVs)

Each row below corresponds to one evidence/*.csv with full 10-column trace. PositionSearch costs are step counts except StayEast (cost .5^x, cost 1.0). Corners and Food use domain costs. Times on Intel x86_64, Python 3.12.3.

CSV	Len	Exp	Time
dfs_tinyMaze	10	15	0.00s
dfs_mediumMaze	130	146	0.01s
dfs_bigMaze	210	390	0.10s
bfs_mediumMaze	68	269	0.08s
bfs_bigMaze	210	620	0.28s

CSV	Len	Exp	Time
ucs_mediumMaze	68	269	0.05s
ucs_testSearch	5	6	0.00s
ucs_stayEast	74	260	0.05s
gbfs_bigMaze_M	210	466	0.13s
astar_bigMaze_0	210	620	0.22s

CSV	Len	Exp	Time
astar_bigMaze_M	210	549	0.21s
dfs_24I3102	42	56	0.01s
bfs_24I3102	24	188	0.03s
ucs_24I3102	24	188	0.03s
gbfs_24I3102_M	28	29	0.00s

CSV	Len	Exp	Time
astar_24I3102_M	24	68	0.01s
bfs_tinyCorners	28	252	0.14s
astar_mediumCorners	106	692	0.86s
astar_trickySearch	60	255	2.55s
closestDot_bigSearch	350	941 rows	0.13s

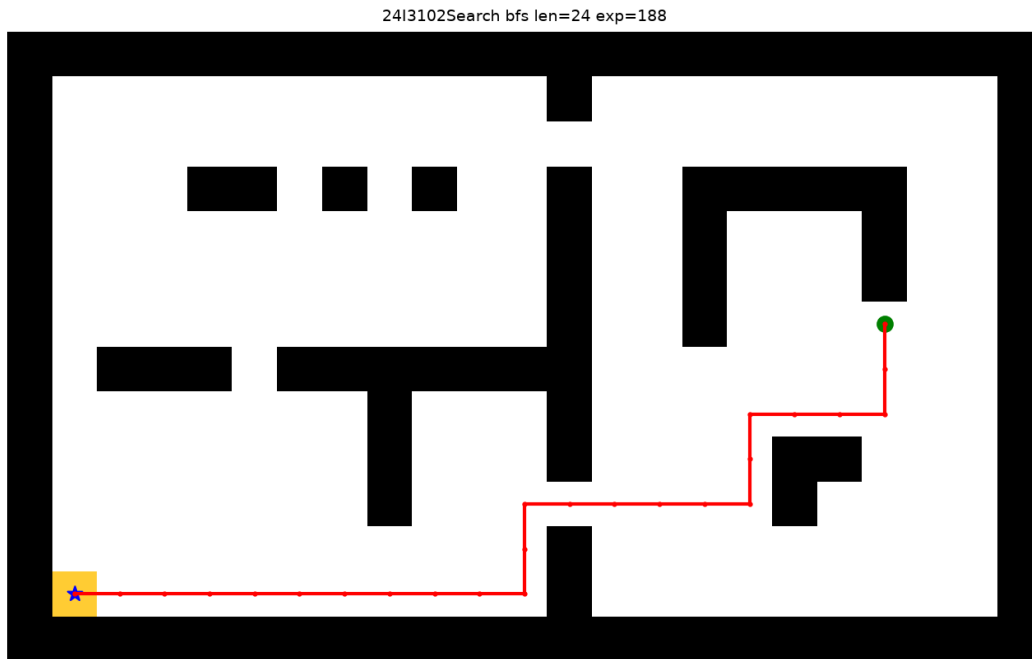
Space/time summary: DFS $O(b^m)$ fastest but longest paths (42 vs 24 custom, 130 vs 68 medium). BFS/UCS $O(b^d)$ optimal

but most expansions (188 vs 68 custom). GBFS fewest expansions (29 custom, 466 big) but suboptimal when deceptive (28 vs 24). A* balances (68 custom, 549 big) and optimal when h consistent. Food MST reduces trickySearch from >15000 (null) to 255.

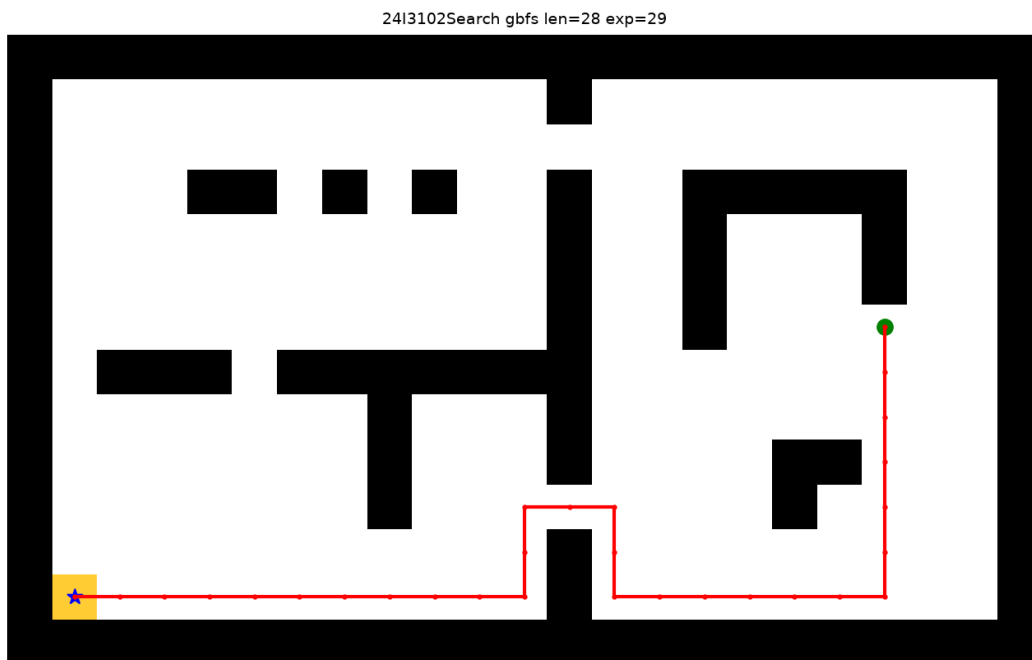
8. Screenshots and visual comparison

Matplotlib renders: black=walls, yellow=food, green=start, blue star=goal, red=path. See evidence/screenshots/. GBFS path on custom maze visibly dips south into winding loop (extra 4 steps); A*/BFS/UCS take north gap direct. DFS wanders deep first. bigMaze control shows identical 210-step corridors (no deception).

Custom BFS optimal 24/188

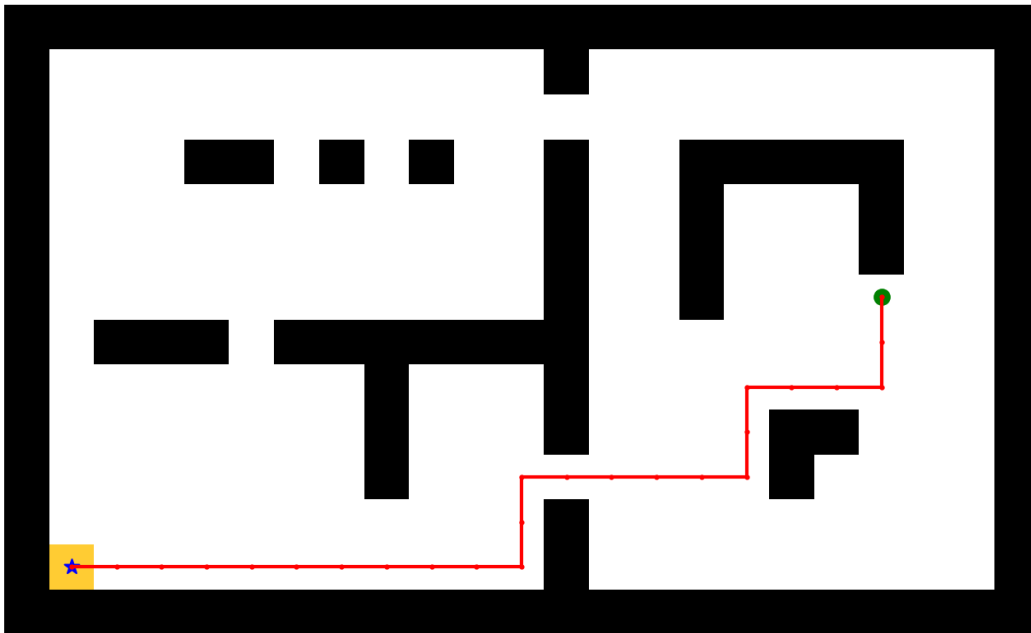


Custom GBFS suboptimal 28/29



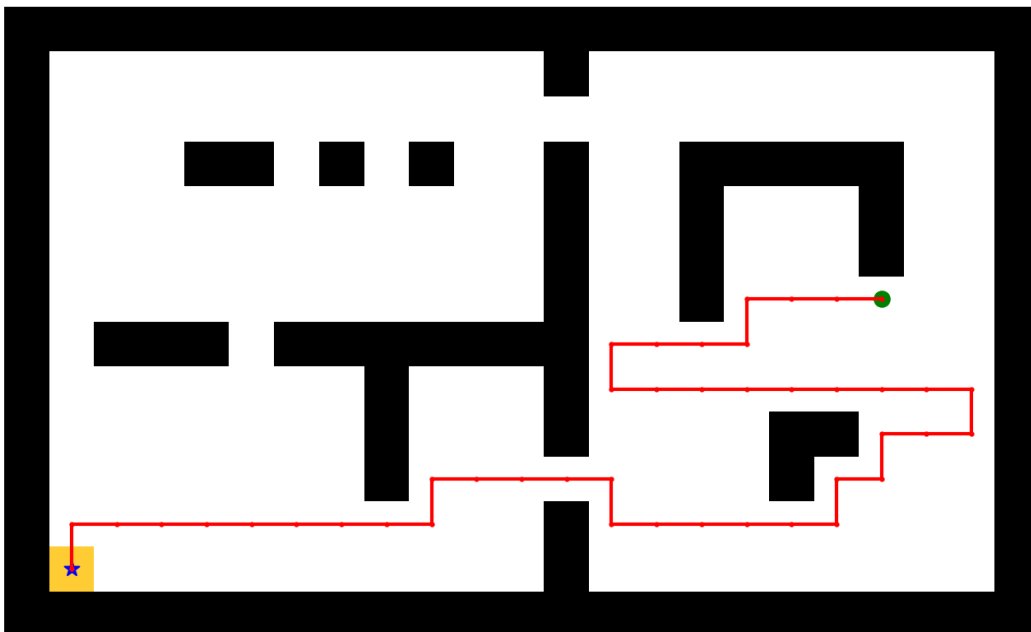
Custom A* optimal 24/68

24I3102Search astar len=24 exp=68



Custom DFS 42/56

24I3102Search dfs len=42 exp=56



9. Conclusion, viva and reproduction

- Reproduce: see README.txt commands; autograder.py --no-graphics; evidence/*.csv header check; layouts/24I3102Search.lay loads (W23 H14).
- Viva line-by-line: Stack push/pop + explored skip (DFS), Queue mark-on-enqueue (BFS), PQ + bestG + update (UCS/A*), h-only push (GBFS), $f=g+h$ (A*), Corners tuple update, MST Prim with cached d, pauseTracing.
- Unseen mazes: algorithms are problem-agnostic (only use SearchProblem API); heuristics use only walls/food/pos, so generalize; food cache is per-problem dict, no leakage.
- Work split: Person1 base search, Person2 searchAgents, Person3 (24I3102) DFS fix, logger, cache (96s to 2.5s), custom maze, docs, packaging.
- Limits: GBFS suboptimal by design; food MST exact-d is strong but still exponential worst-case; DFS path 42 vs optimal 24 on custom shows depth risk.

Submission ZIP contains full project, edited search.py/searchAgents.py, original core files, README.txt, this report.pdf,

layouts/*.lay + 24I3102Search.lay, evidence/*.csv (20) + screenshots/*.png (7). Bring printed rubric to viva.